# Giới thiệu về bảo mật API

NGƯỜI TRÌNH BÀY: NGUYỄN HỮU TRUNG

# Tại sao cần bảo mật API?

API là cầu nối giữa ứng dụng và dữ liệu, nếu không được bảo vệ đúng cách, hacker có thể khai thác lỗ hổng để xâm nhập trái phép và chiếm quyền điều khiển toàn bộ hệ thống.

**Nguy cơ tấn công API**

API không bảo mật dễ bị hacker khai thác gây mất kiểm soát và rò rỉ dữ liệu nhạy cảm.

**Các kiểu tấn công phổ biến**

Brute-force, DDoS và XSS là các cuộc tấn công thường nhắm vào API thiếu bảo mật.

**Lợi ích của bảo mật API**

Bảo mật API ngăn truy cập trái phép, xác thực người dùng và giảm thiểu rủi ro cho hệ thống.

**Phòng thủ đa lớp**

Kết hợp nhiều lớp bảo mật giúp hệ thống phòng thủ tốt và bảo vệ uy tín doanh nghiệp.

# Tổng quan 4 lớp bảo mật

**Xác thực đầu vào**

Input Validation làm sạch và kiểm tra dữ liệu để ngăn chặn dữ liệu sai lệch và tấn công XSS.

**Giới hạn tần suất truy cập**

Rate Limiting hạn chế số lần truy cập để chống brute-force và DDoS, bảo vệ máy chủ khỏi quá tải.

**Xác thực người dùng**

Authentication xác định danh tính người dùng, bảo vệ dữ liệu nhạy cảm bằng cách trả lời câu hỏi “Bạn là ai?”.

**Phân quyền người dùng**

Authorization kiểm soát quyền truy cập, trả lời câu hỏi “Bạn được phép làm gì?” sau khi đăng nhập.

# Chi tiết các lớp bảo mật

# Input Validation (Xác thực dữ liệu)

### Vai trò của Input Validation

Input Validation đảm bảo dữ liệu người dùng gửi lên luôn hợp lệ và an toàn trước khi xử lý.

### Kiểm tra Backend cần thiết

Xác thực dữ liệu phải được thực hiện ở backend để ngăn chặn các tấn công qua công cụ như Postman.

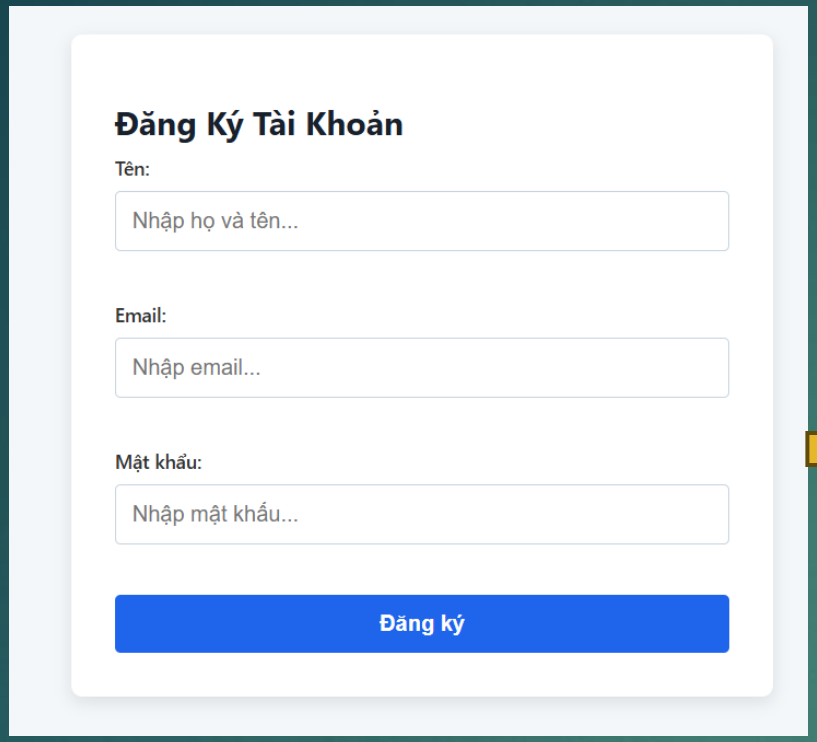
### Quy tắc và Thư viện

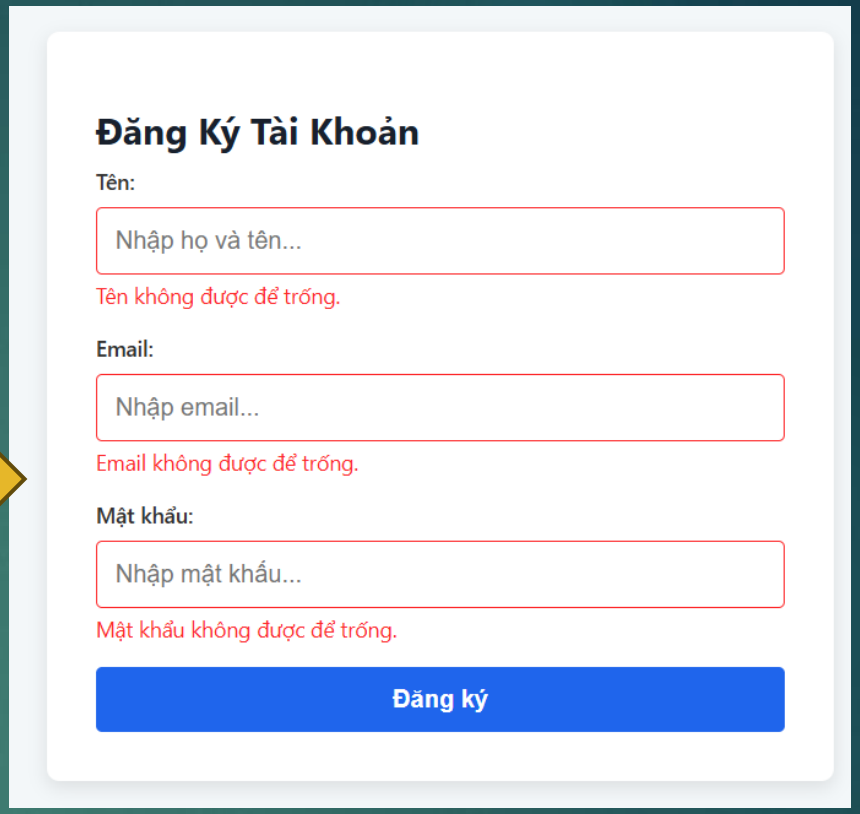
Sử dụng các thư viện như express-validator để định nghĩa quy tắc validate và sanitize dữ liệu đầu vào.

### Nguyên tắc bảo mật quan trọng

Không bao giờ tin tưởng dữ liệu từ frontend, luôn validate và sanitize tại backend để bảo vệ hệ thống.

# Ví dụ

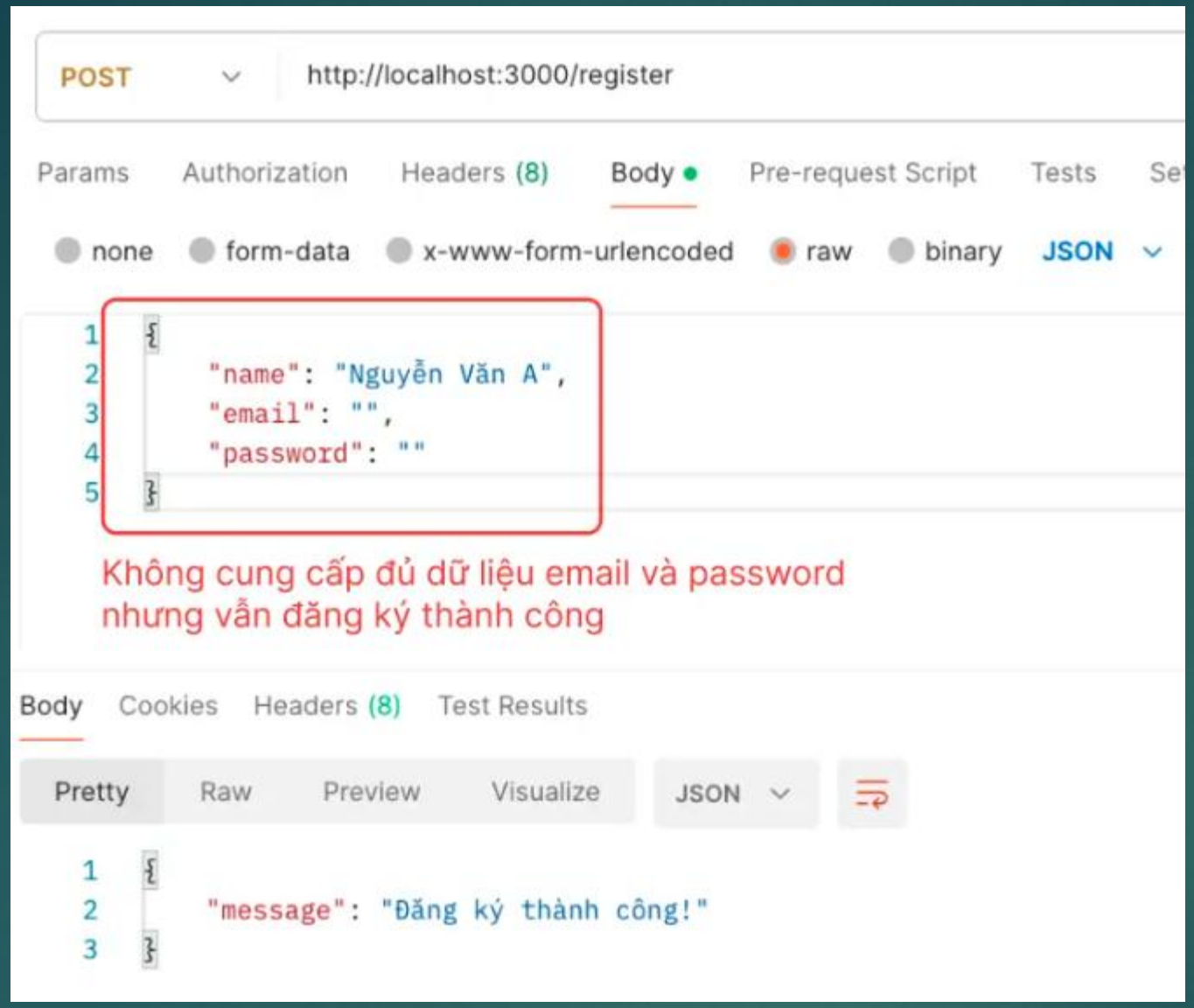



- FrontEnd đã có cơ chế xác thực, yêu cầu người dùng phải nhập đủ thông tin.

```
// Real-time validation
nameInput.addEventListener('input', () => clearError(nameInput, nameError));
emailInput.addEventListener('input', () => clearError(emailInput, emailError));
passwordInput.addEventListener('input', () => clearError(passwordInput, passwordError));

// Email validation regex
const isEmailValid = (email) => {
    const re = /^(([^<>()[\]\\.,;:\s@"]+(\.[^<>()[\]\\.,;:\s@"]+)*)|(".+"))@((\[[0-9]{1,3}\.[0-9]{1,3}\.[0-
9]{1,3}\.[0-9]{1,3}\])|(([a-zA-Z\-0-9]+\.)+[a-zA-Z]{2,}))$/;
    return re.test(String(email).toLowerCase());
};
```

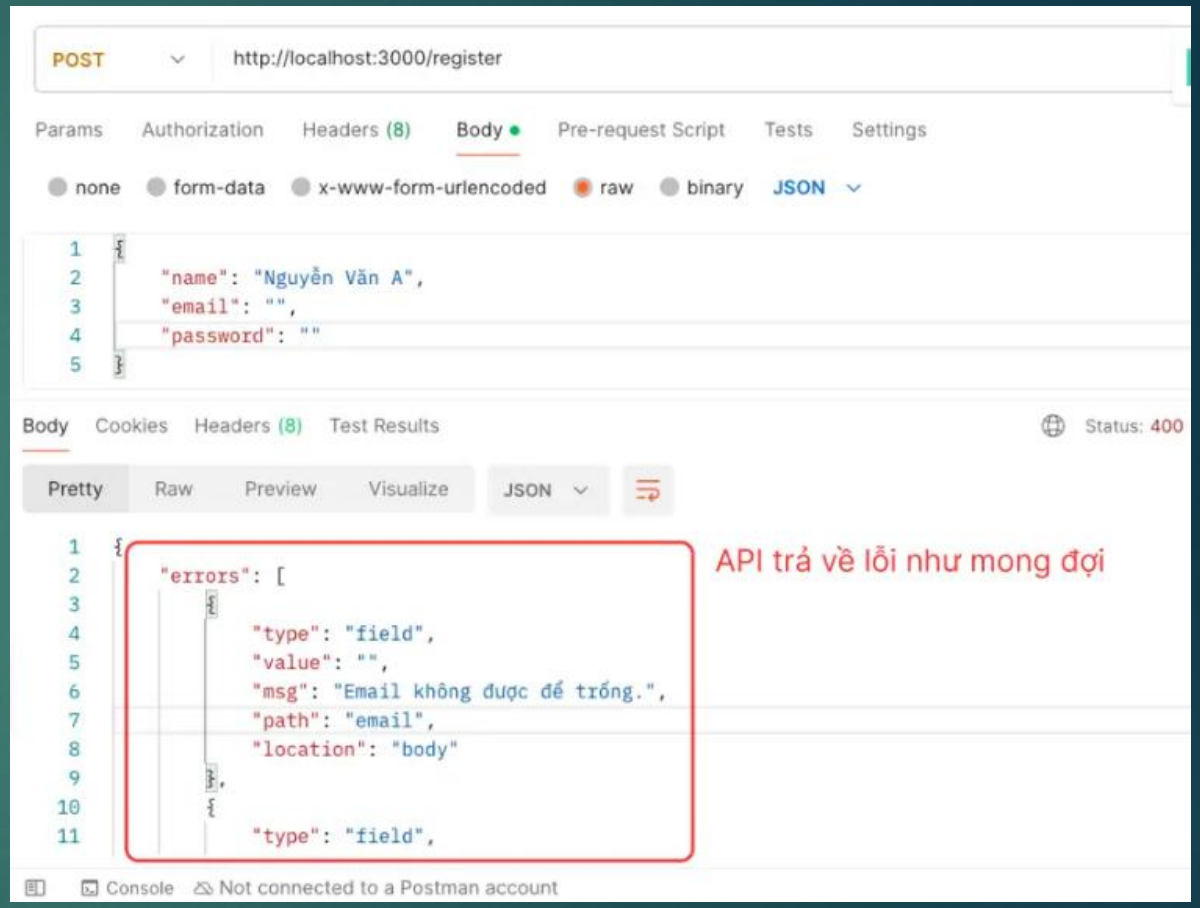
# Ví dụ




- sử dụng Postman để gọi thẳng vào API đăng ký và cố tình không cung cấp đầy đủ dữ liệu.

# Ví dụ

```
"express-validator": "^7.0.1",

==========================

const { body, validationResult } = require('express-validator');

=================================

// Định nghĩa các quy tắc xác thực dữ liệu
const registerValidationRules = [
  body('name').escape().not().isEmpty().withMessage('Tên không được để trống.'),
  body('email').not().isEmpty().withMessage('Email không được để trống.'),
  body('password').not().isEmpty().withMessage('Password không được để trống.'),
];
==================================

// API Endpoint để đăng ký người dùng
app.post('/register', registerValidationRules, async (req, res) => {
  // Kiểm tra kết quả của validation
  const errors = validationResult(req);
```



# Rate Limiting (Giới hạn tần suất truy cập)

### Ngăn chặn tấn công quá tải

Rate Limiting giúp chống lại các cuộc tấn công brute-force và DDoS bằng cách giới hạn số yêu cầu gửi đến API trong thời gian ngắn.

### Thiết lập giới hạn truy cập

Bạn có thể cấu hình giới hạn như 5 yêu cầu mỗi IP trong 15 phút sử dụng thư viện express-rate-limit để kiểm soát lưu lượng truy cập.

### Đảm bảo hoạt động ổn định

Khi vượt quá giới hạn, hệ thống trả về mã lỗi và chặn các yêu cầu tiếp theo, giữ cho máy chủ vận hành ổn định và hiệu quả.

### Giảm rủi ro bảo mật

Rate Limiting không chỉ bảo vệ hiệu suất mà còn ngăn chặn khai thác lỗ hổng bảo mật trên các API nhạy cảm như đăng nhập.

# Ví dụ

- Đây là một đoạn script mô phỏng việc gọi API đăng nhập 20,000 lần với các thông tin email và password ngẫu nhiên. Khi script này chạy, máy chủ gần như chắc chắn sẽ bị quá tải ngay lập tức.

```
  echo "----------------------------------------"
  exit 0 # Thoát script ngay khi tìm thấy mật khẩu
 elif [ "$response" -eq 429 ]; then
  echo "!!! BỊ CHẶN! API đã áp dụng Rate Limiting (Mã trạng thái: $response). Các yêu cầu tiếp theo
từ IP này sẽ bị chặn."
 else
  echo "Thất bại với mã trạng thái: $response"
 fi

 # Thêm một khoảng nghỉ nhỏ để dễ quan sát
 sleep 0.5
done

echo "Demo hoàn tất. Đã cho thấy cơ chế Rate Limiting hoạt động."
```

```
#!/bin/bash

# URL của API đăng nhập mục tiêu
LOGIN_URL="http://localhost:3000/login"

# Tên người dùng/email không đổi
EMAIL="trungnh@iotstar.vn"

# Vòng lặp để thử 20000 mật khẩu giả định
for i in {1..20000}
do
 # Tạo mật khẩu giả định cho lần thử thứ i
 PASSWORD="password$i"

 echo "Đang thử lần thứ $i với mật khẩu: $PASSWORD"

 # Sử dụng curl để gửi yêu cầu POST với định dạng JSON
 response=$(curl -s -o /dev/null -w "%{http_code}" -X POST $LOGIN_URL \
     -H 'Content-Type: application/json' \
     -d "{\"email\":\"$EMAIL\", \"password\":\"$PASSWORD\"}")

 # Kiểm tra mã trạng thái HTTP trả về
 if [ "$response" -eq 200 ]; then
  echo "----------------------------------------"
  echo "THÀNH CÔNG! Mật khẩu là: $PASSWORD"
```

# Ví dụ

- Cấu hình Rate Limiting cho API đăng nhập, ví dụ bằng cách sử dụng thư viện express-rate-limit, mình có thể thiết lập quy tắc đơn giản như sau:

```
// API Endpoint để đăng ký người dùng
app.post('/register', apiLimiter, registerValidationRules, async (req, res) => {
  // Kiểm tra kết quả của validation
  const errors = validationResult(req);
```

```
 "express-rate-limit": "^8.1.0",
==========================
const rateLimit = require('express-rate-limit');
==================
// Cấu hình rate limit
const apiLimiter = rateLimit({
  windowMs: 10 * 60 * 1000, // 10 phút
  max: 5, // Giới hạn mỗi IP 5 request trong 1 khoảng thời gian (windowMs)
  message: 'Quá nhiều yêu cầu từ IP này, vui lòng thử lại sau 10 phút.',
  standardHeaders: true,
  legacyHeaders: false,
});
```

```
    echo "----------------------------------------"
    exit 0 # Thoát script ngay khi tìm thấy mật khẩu
  elif [ "$response" -eq 429 ]; then
    echo "!!! BỊ CHẶN! API đã áp dụng Rate Limiting (Mã trạng thái: $response). Các yêu cầu tiếp theo
từ IP này sẽ bị chặn."
  else
    echo "Thất bại với mã trạng thái: $response"
  fi

  # Thêm một khoảng nghỉ nhỏ để dễ quan sát
  sleep 0.5
done

echo "Demo hoàn tất. Đã cho thấy cơ chế Rate Limiting hoạt động."
```

# Authentication (Xác thực)

### Khái niệm xác thực

Authentication xác nhận danh tính người dùng để chỉ những người hợp lệ mới được truy cập dữ liệu nhạy cảm.

### Vai trò của JWT

JWT tạo token sau đăng nhập, dùng để xác nhận và bảo vệ truy cập API bằng cách kiểm tra token hợp lệ.

### Middleware kiểm tra bảo mật

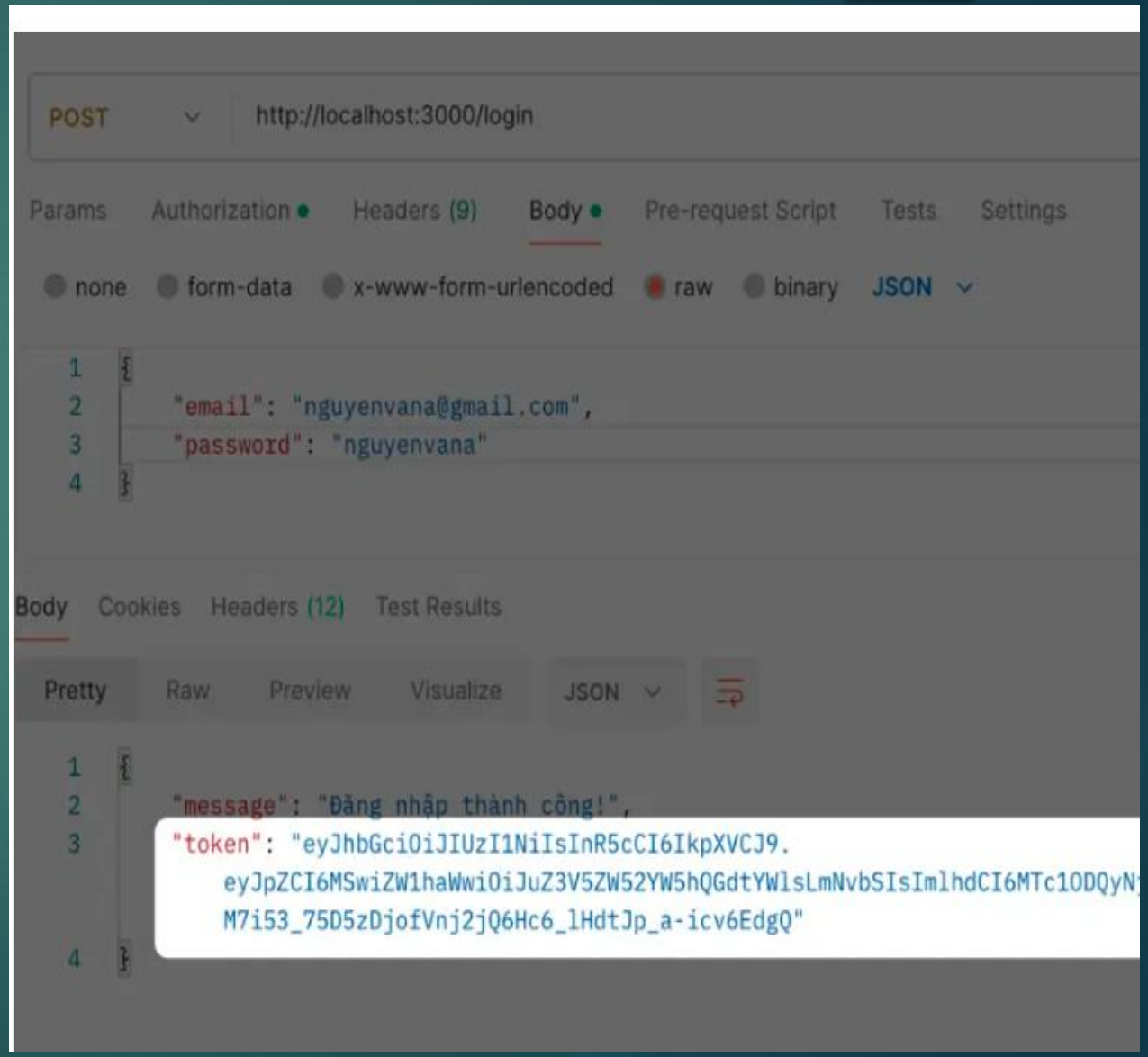
Middleware kiểm tra token hợp lệ và còn hiệu lực trước khi cho phép truy cập tài nguyên nhạy cảm.

### Tầm quan trọng của xác thực

Xác thực là lớp bảo mật bắt buộc nhằm bảo vệ quyền riêng tư và ngăn truy cập trái phép vào hệ thống.

# Ví dụ

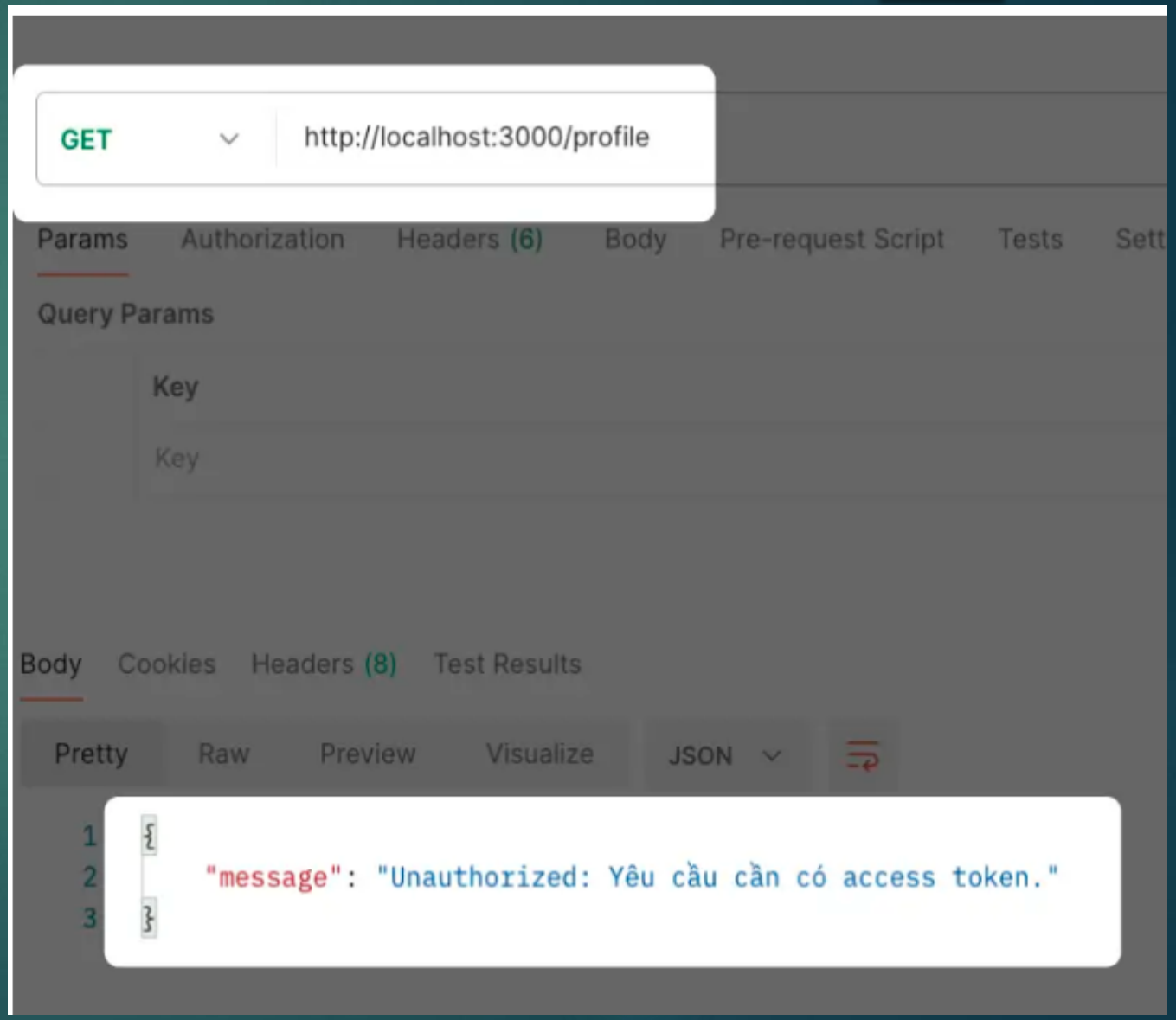
- gọi API /login với email và password chính xác để hệ thống trả về một token:



# Ví dụ

- Gọi API /profile mà không đính kèm token. Bạn có thể thấy, hệ thống sẽ báo lỗi ngay lập tức.

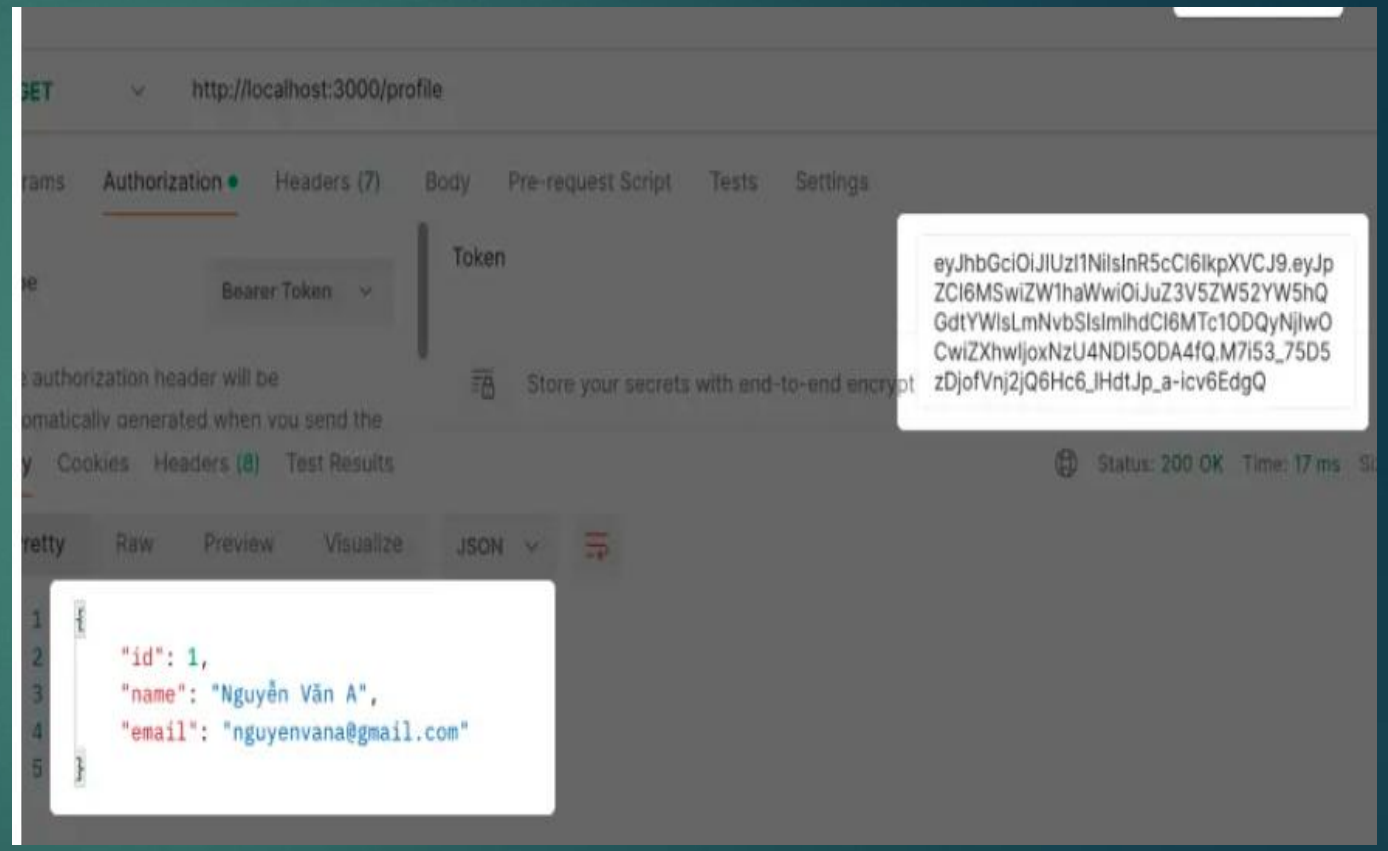
```
// Middleware để xác thực token
function authenticateToken(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1]; // Lấy token từ header "Bearer <token>"

  if (token == null) {
    return res.status(401).json({ message: 'Unauthorized: Yêu cầu cần có access token.' });
  }

  jwt.verify(token, JWT_SECRET, (err, user) => {
    if (err) {
      return res.status(403).json({ message: 'Forbidden: Token không hợp lệ hoặc đã hết hạn.' });
    }
    req.user = user; // Lưu thông tin người dùng đã xác thực vào request
    next(); // Đi tiếp tới route handler
  });
}

// API Endpoint để lấy thông tin profile (yêu cầu xác thực)
app.get('/profile', authenticateToken, async (req, res) => {
  try {
    // req.user được gán từ middleware authenticateToken
    const user = await User.findOne({ where: { id: req.user.id } });
```



# Ví dụ

- Nhưng nếu mình đính kèm token vào Header, API sẽ trả về đúng thông tin người dùng

```
  // Trả về thông tin người dùng (không bao gồm mật khẩu)
  res.status(200).json({
    id: user.id,
    name: user.name,
    email: user.email,
  });
 } catch (error) {
  console.error('❌ Lỗi khi lấy thông tin profile:', error);
  res.status(500).json({ errors: [{ msg: 'Lỗi server, không thể lấy thông tin.' }] });
 }
});
```



# Authorization (Phân quyền)

**Định nghĩa phân quyền**

Authorization xác định quyền hạn của người dùng sau khi đã xác thực, trả lời câu hỏi bạn được phép làm gì.

**Phân quyền dựa trên vai trò**

Phân quyền được thiết lập dựa trên vai trò như admin có quyền xóa, user chỉ được xem hoặc chỉnh sửa dữ liệu.

**Kiểm tra quyền với middleware**

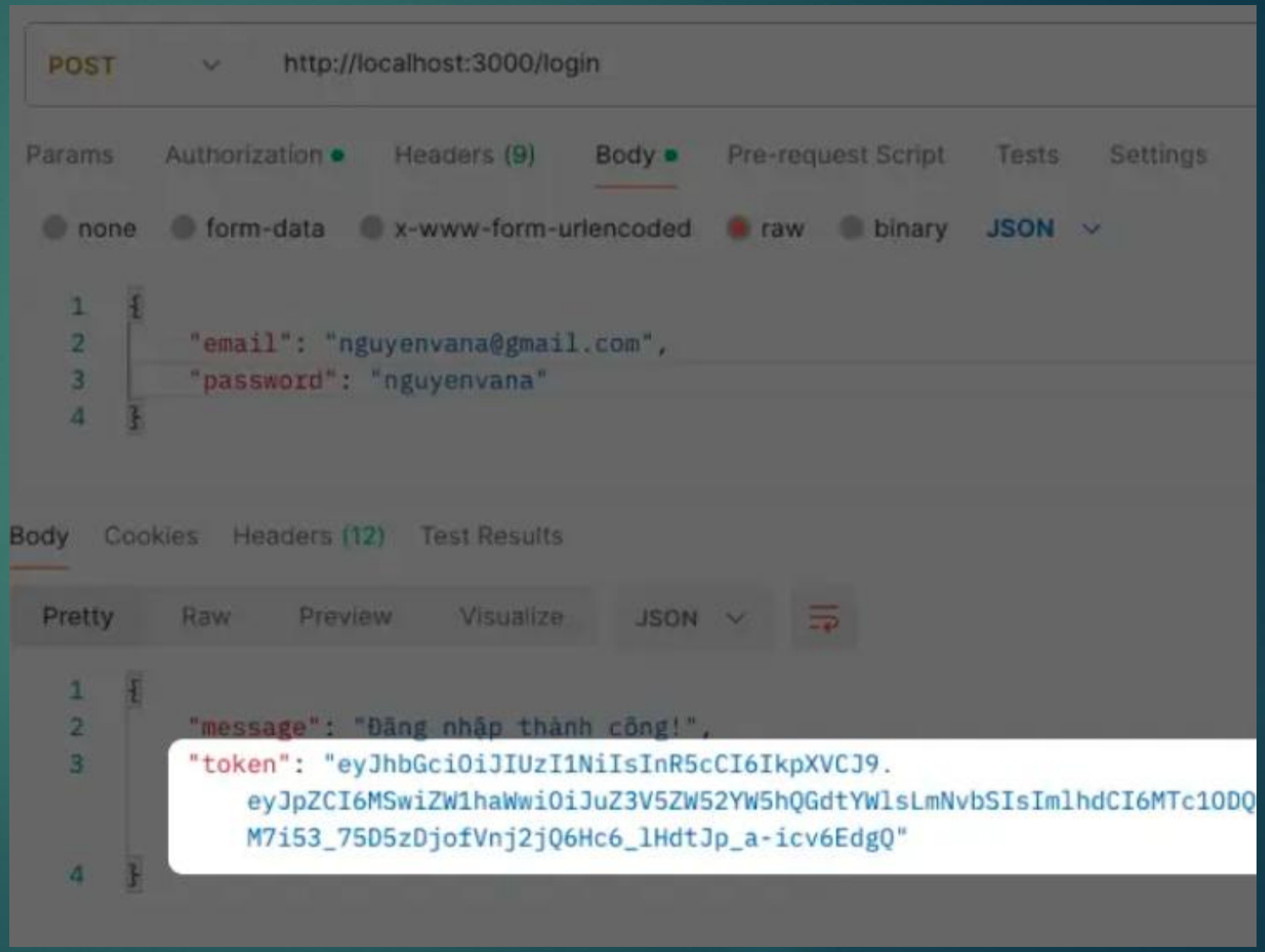
Middleware kiểm tra quyền trước khi thực hiện hành động, ví dụ API DELETE chỉ cho phép admin thực hiện.

**Vai trò của Authorization trong bảo mật**

Authorization là lớp bảo mật thứ hai sau Authentication, giúp ngăn truy cập trái phép và giảm thiểu thiệt hại.

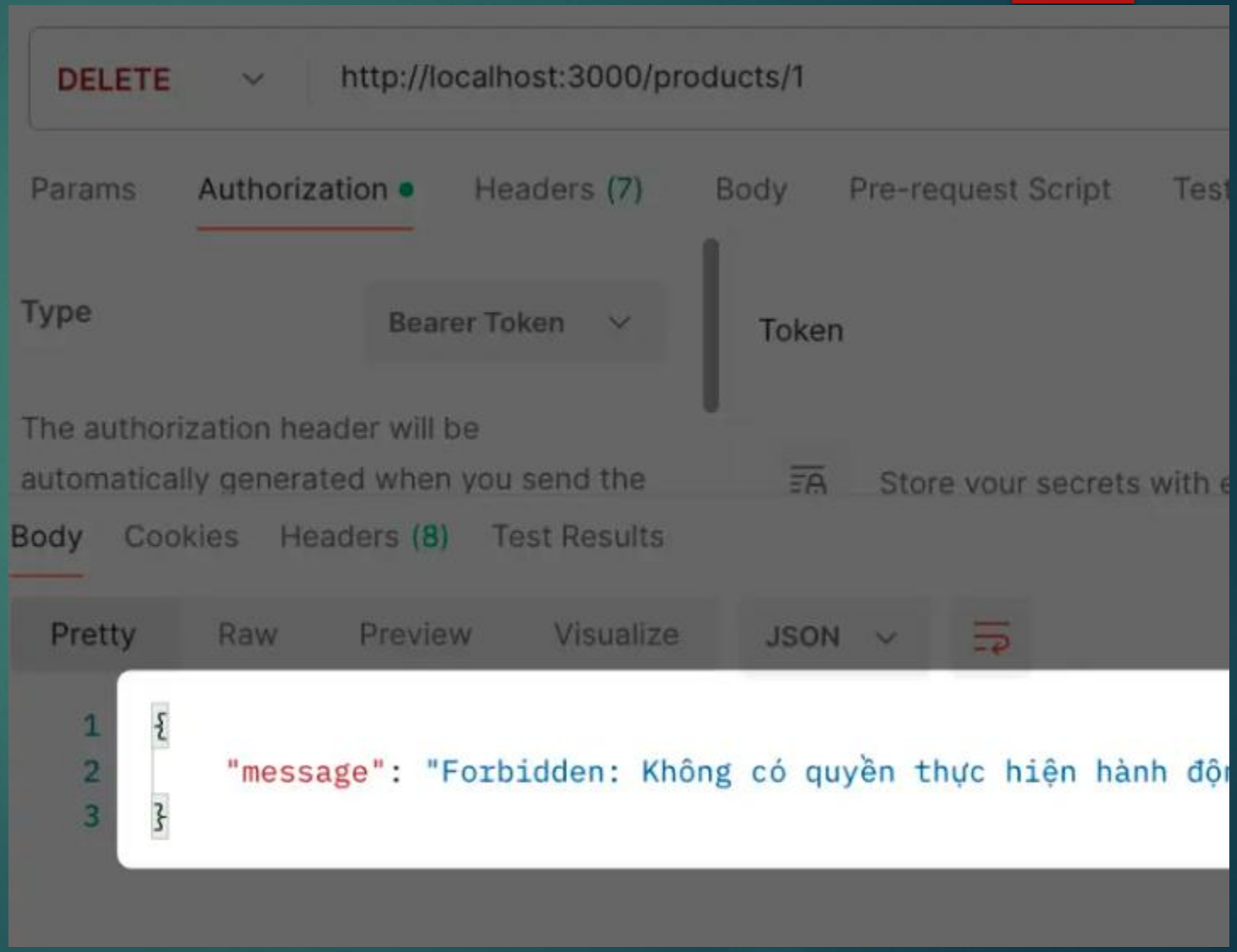
# Ví dụ

- **Trường hợp 1: Đăng nhập với vai trò User**
  - Đầu tiên, đăng nhập bằng tài khoản user và nhận về một token



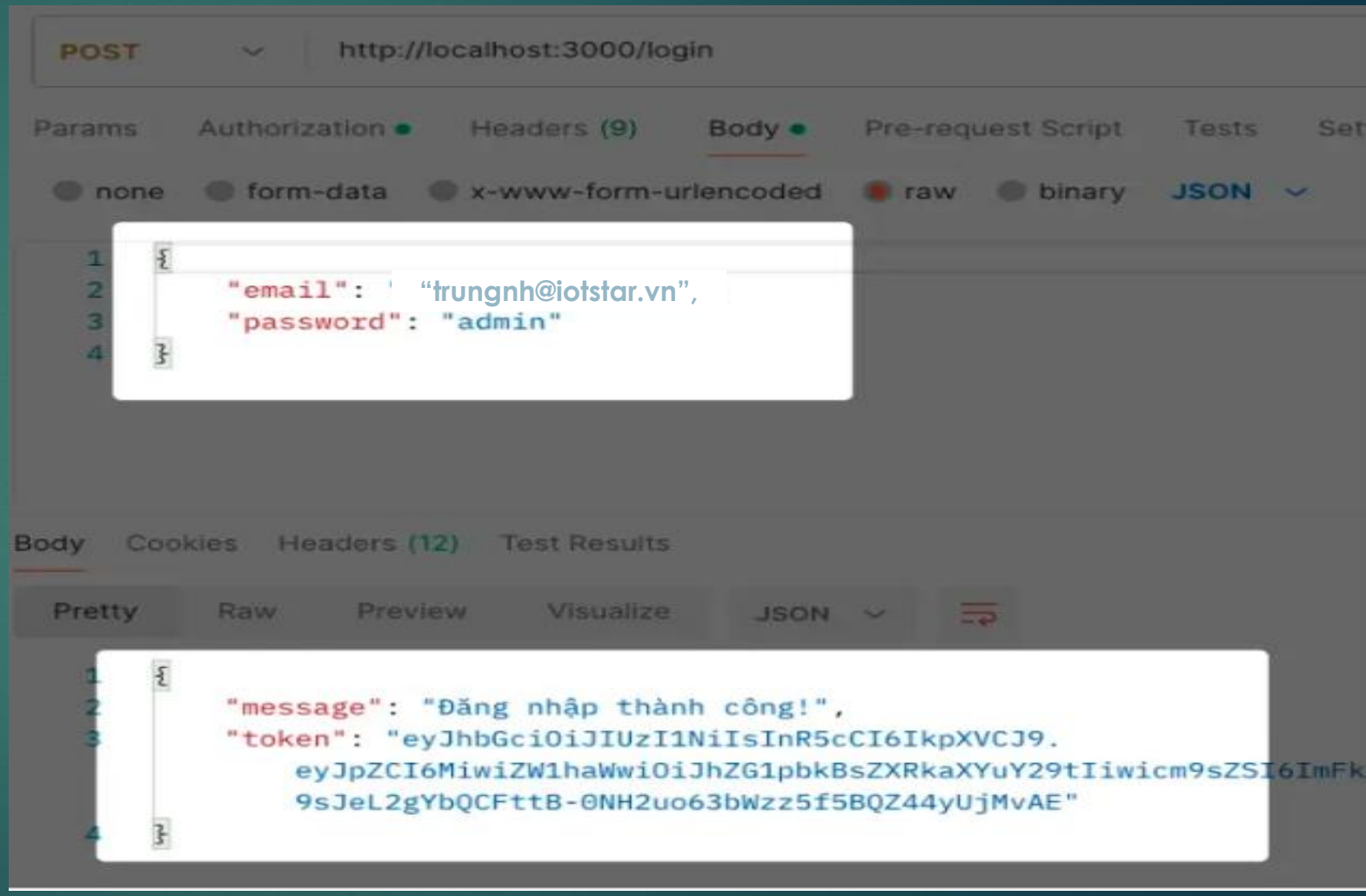
# Ví dụ

- **Trường hợp 1: Đăng nhập với vai trò User**
  - Sau đó, dùng token này để gọi API xóa sản phẩm. Hệ thống sẽ từ chối và phản hồi về lỗi "Không có quyền thực hiện".



# Ví dụ

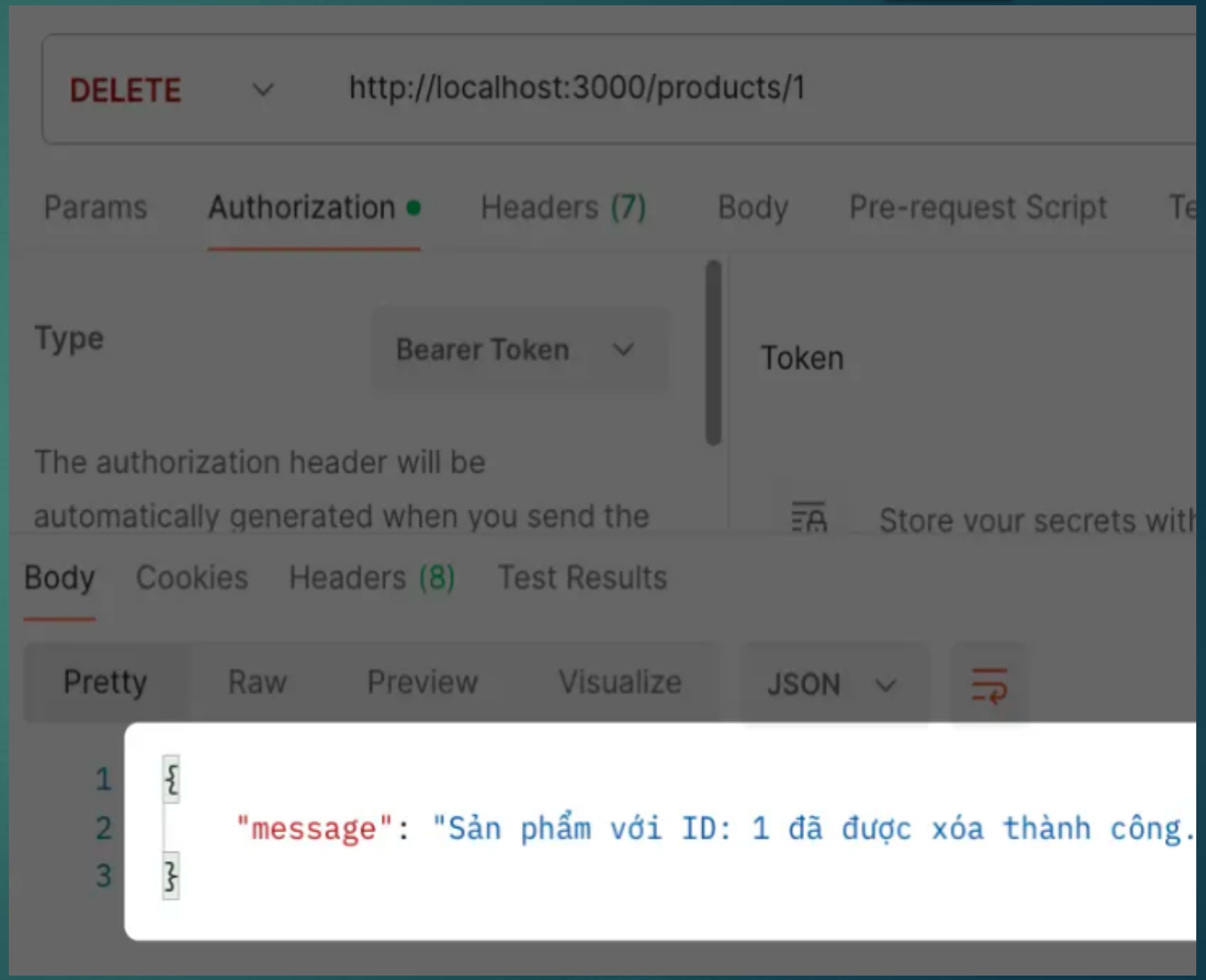
- **Trường hợp 2: Đăng nhập với vai trò Admin**
  - Đầu tiên, đăng nhập bằng tài khoản admin và nhận về một token



# Ví dụ

- **Trường hợp 2: Đăng nhập với vai trò Admin**
  - Sau đó, Khi dùng token này để gọi lại API xóa sản phẩm, hệ thống đã thực hiện thành công.

```
// Middleware để phân quyền dựa trên vai trò (role)
function authorize(role) {
  return (req, res, next) => {
    if (req.user.role !== role) {
      return res.status(403).json({ message: 'Forbidden: Không có quyền thực hiện hành động này.' });
    }
    next();
  };
}

// API Endpoint để xóa sản phẩm (yêu cầu quyền Admin)
app.delete('/products/:id', authenticateToken, authorize('admin'), (req, res) => {
```



# Tổng kết và nguyên tắc vàng

# Nguyên tắc bảo mật API

**Xác thực và làm sạch dữ liệu**

Luôn validate và sanitize dữ liệu tại backend để không tin tưởng frontend, tránh lỗ hổng bảo mật.

**Phân quyền tối thiểu**

Cấp quyền cần thiết cho từng vai trò nhằm giảm thiểu rủi ro bảo mật từ các truy cập không hợp lệ.

**Bảo mật nhiều lớp**

Kết hợp Authentication và Authorization tạo hệ thống phòng thủ nhiều lớp, tăng cường bảo mật API.

**Cập nhật thư viện thường xuyên**

Sử dụng phiên bản thư viện mới nhất để tránh lỗ hổng bảo mật từ phần mềm cũ lỗi thời.